

Combined_ResNet_hqnn

May 8, 2026

1 ResNet Classifier - Make, Model, Year

Combining VMRRdb and StanfordCars into a single dataset to train on.

1.0.1 Hyperparameters

```
[25]: # Num of images per class
MIN_CLASSES = 40
MAX_CLASSES = 100

# batch size
BATCH_SIZE = 16

# Number of iterations over dataset
epochs = 60
```

1.1 Imports

```
[ ]: # PyTorch
import torchvision.transforms as tt
import torchvision.models as models
from torch.utils.data import Dataset
from torch.utils.data import ConcatDataset
from torchvision.datasets.folder import default_loader
from torch.utils.data import TensorDataset
from torch.utils.data import DataLoader
from torch.utils.data import Subset
import torch.nn.functional as F
import torch.nn as nn
import torch.optim as optim
import torch
import pennylane as qml

# Utility
from collections import Counter, defaultdict
import scipy.io as sio
import pathlib
```

```

import os
import re
from PIL import Image
from tqdm import tqdm
import kagglehub
import random
import matplotlib.pyplot as plt
import time

```

1.2 StanfordCars Setup

1.2.1 Download Dataset

```

[ ]: path_stanford_cars = kagglehub.dataset_download("eduardo4jesus/
↳stanford-cars-dataset")

print("Path to dataset files:", path_stanford_cars)

```

Path to dataset files:

C:\Users\godca\.cache\kagglehub\datasets\eduardo4jesus\stanford-cars-dataset\versions\1

1.2.2 StanfordCars Dataset Definition

Stanford dataset class that's been modified from the PyTorch implementation

```

[ ]: class StanfordCars(Dataset):
    def __init__(self, root, global_class_to_idx, transform=None,
↳max_images_per_class=None, class_normalizer=None):
        self.root = pathlib.Path(root)
        self.class_to_idx = global_class_to_idx
        self.loader = default_loader
        self.transform = transform
        self.class_normalizer = class_normalizer
        self.class_counts = Counter()

        # Paths
        devkit = self.root / "car_devkit/devkit"
        ann_path = devkit / "cars_train_annos.mat"
        img_dir = self.root / "cars_train/cars_train"

        # Loading metadata from matlab files
        annotations = sio.loadmat(ann_path, squeeze_me=True)["annotations"]
        raw_classes = sio.loadmat(devkit / "cars_meta.mat",
↳squeeze_me=True)["class_names"].tolist()

        self.samples = []

```

```

        # Building the dataset list, images labeled with their corresponding
        ↪normalized class labels
        for ann in annotations:
            path = str(img_dir / ann["fname"])
            cls = raw_classes[ann["class"] - 1]

            # Normalized class
            if class_normalizer:
                cls = class_normalizer(cls)

            idx = self.class_to_idx.get(cls, None)
            if idx is None:
                continue

            # Skip images after max number of images per class has been hit
            if max_images_per_class and self.class_counts[cls] >=
        ↪max_images_per_class:
                continue

            # Grouping image paths with their new class label
            self.samples.append((path, idx))
            self.class_counts[cls] += 1

    def __len__(self):
        return len(self.samples)

    # Given an index returns the corresponding transformed image, along with
    ↪their class label
    def __getitem__(self, idx):
        path, y = self.samples[idx]
        img = self.loader(path)
        if self.transform:
            img = self.transform(img)
        return img, y

```

1.3 VMRRdb Setup

1.3.1 Dataset Download

```

[29]: path_vmmrdb = kagglehub.dataset_download("prabashwara/vmmrdb-dataset")

print("Path to dataset files:", path_vmmrdb)

```

Path to dataset files:

C:\Users\godca\.cache\kagglehub\datasets\prabashwara\vmmrdb-dataset\versions\1

1.3.2 Custom VMNRDB Dataset Definition

Defining a PyTorch dataset class for easy access to the image data using PyTorch DataLoaders.

```
[ ]: class VMNRDB_Dataset(Dataset):
    def __init__(self, root_dir, global_class_to_idx, class_normalization,
        ↪max_images_per_class=None, transform=None):
        self.root_dir = root_dir
        self.transform = transform
        self.image_paths = []
        self.labels = []
        self.class_to_idx = global_class_to_idx
        self.classes = []
        self.class_counts = Counter()

        # Images to class names
        # Class names are in the folder names
        for class_name in sorted(os.listdir(root_dir)):
            class_path = os.path.join(root_dir, class_name)
            if os.path.isdir(class_path):

                # Normalizing the class names and getting the index for said class
                stripped = class_normalization(class_name)
                idx = self.class_to_idx.get(stripped, None)
                if idx is None:
                    continue

                # Labeling all images with their corresponding classes +
                # skipping images after max number of images per class has been hit
                for img_name in os.listdir(class_path):
                    if img_name.lower().endswith(('.png', '.jpg', '.jpeg')):
                        if max_images_per_class and self.class_counts[stripped] >=
        ↪max_images_per_class:
                            continue
                        self.image_paths.append(os.path.join(class_path, img_name))
                        self.labels.append(idx)
                        self.class_counts[stripped] += 1

        self.classes = self.class_to_idx.keys()

        # Overridden length function
        def __len__(self):
            return len(self.image_paths)

        # Given an index, retrieves the transformed
        # image and corresponding class label
        def __getitem__(self, idx):
```

```

img_path = self.image_paths[idx]
label = self.labels[idx]

image = Image.open(img_path).convert('RGB')
if self.transform:
    image = self.transform(image)

return image, label

```

1.4 Dataset Combination

1.4.1 Normalization Function

This function is used to normalize the class labels in both the VMMDb dataset and StanfordCars dataset

```

[ ]: # All components that show up in class names that aren't part of model or make
EXTRA = {
    'sedan', 'coupe', 'convertible', 'hatchback', 'suv', 'wagon', 'van',
    'minivan', 'pickup', 'truck', 'cab', 'crew', 'extended', 'regular',
    'quad', 'club', 'cargo', 'passenger', 'sut', 'roadster', 'hybrid',
    'conv.', 'drophead', 'supercab', 'classic', 'superleggera',
    'abarth', 'activehybrid', 'series', 'electric',
}

# Normalizes classes in both datasets
def normalize_classes(car_class):
    car_class = re.sub('_', ' ', car_class).lower()
    car_class = re.sub("-", "", car_class)
    car_class_components = car_class.split(" ")

    # Removing all parts of the class not corresponding to model, make and year
    car_class_components = [t for t in car_class_components if t not in EXTRA]

    # Normalizing some more inconsistencies between the two datasets
    car_class_components = ["mercedes benz" if t == "mercedesbenz" else t for t in car_class_components]
    car_class_components = ["rolls royce" if t == "rollsroyce" else t for t in car_class_components]

    return " ".join(car_class_components)

```

1.4.2 Combining the two datasets

Using a minimum number of 40 images and a maximum number of 100 per class.

```

[32]: # Building global class_to_idx so we can combine the two datasets properly

```

```

def build_global_class_to_idx(stanford_root, vmrdb_root, normalize_classes,
    min_classes):
    class_counts = Counter()

    # Stanford Cars
    devkit = pathlib.Path(stanford_root) / "car_devkit/devkit"
    ann_path = devkit / "cars_train_annos.mat"
    annotations = sio.loadmat(ann_path, squeeze_me=True)["annotations"]
    raw_classes = sio.loadmat(devkit / "cars_meta.mat",
    squeeze_me=True)["class_names"].tolist()

    # Normalizing class names + retrieving image count per class
    for ann in annotations:
        cls = normalize_classes(raw_classes[ann["class"] - 1])
        class_counts[cls] += 1

    # VMRRdb
    for class_name in sorted(os.listdir(vmrdb_root)):
        class_path = os.path.join(vmrdb_root, class_name)
        if os.path.isdir(class_path):

            # Normalizing class names + retrieving image count per class
            cls = normalize_classes(class_name)
            for img in os.listdir(class_path):
                if img.lower().endswith((".jpg", ".jpeg", ".png")):
                    class_counts[cls] += 1

    # Removing classes with less than min_classes number of images
    trimmed_classes = {cls for cls, count in class_counts.items() if count >=
    min_classes}
    class_to_idx = {cls : idx for idx, cls in
    enumerate(sorted(trimmed_classes))}
    class_counts = {cls : count for cls, count in class_counts.items() if cls in
    trimmed_classes}

    return class_to_idx, class_counts

# Building the global class to idx and finding the number of images per class +
# removes classes with less than MIN_CLASSES images
global_class_to_idx, class_counts =
    build_global_class_to_idx(path_stanford_cars, path_vmrdb,
    normalize_classes, MIN_CLASSES)

```

1.4.3 Defining Training and Test Transforms

Image Normalization Calculation Run the following cell if MIN_CLASSES or MAX_CLASSES is modified or the dataset is modified to get the new pixel mean and std. Then replace mean and std in the cell following this one with the new mean and std.

```
[33]: transform = tt.Compose([
        tt.Resize((224,224)),
        tt.ToTensor(),
    ])
    # Full dataset used to calculate pixel mean and std
    stanford_dataset = StanfordCars(
        root=path_stanford_cars,
        global_class_to_idx=global_class_to_idx,
        class_normalizer=normalize_classes,
        max_images_per_class = MAX_CLASSES,
        transform=transform
    )
    vmrdb_dataset = VMRDB_Dataset(
        root_dir=path_vmrdb,
        global_class_to_idx=global_class_to_idx,
        class_normalization=normalize_classes,
        max_images_per_class = MAX_CLASSES,
        transform=transform
    )
    full_dataset = ConcatDataset([stanford_dataset, vmrdb_dataset])
    data_loader = DataLoader(full_dataset, BATCH_SIZE, num_workers = 0, pin_memory_
        ↪ = True, shuffle=False)
```

```
[ ]: # Mean and Standard Deviation Calculation
mean = 0
ex2 = 0
total_pixels = 0

loop = tqdm(data_loader)
for images, _ in loop:
    B, C, H, W = images.shape

    images = images.view(B, C, -1) # H x W

    mean += images.sum(dim=(0, 2))
    ex2 += (images ** 2).sum(dim=(0, 2))

    total_pixels += B * H * W

    loop.set_description("mean + std calculation")

mean /= total_pixels
```

```

ex2 /= total_pixels

std = torch.sqrt(ex2 - mean ** 2)

print()
print("Mean:", mean)
print("Std:", std)

```

Dataset Transform definitions Separating the transform definitions for the test and train sets, with the training dataset using data augmentation.

Image normalization is used to help speed up convergence and stabilize training.

```

[ ]: # Mean and standard deviation values found above
mean = [0.4369, 0.4330, 0.4285]
std = [0.2681, 0.2649, 0.2673]

# Defines image preprocessing for the training and test datasets
train_transform = tt.Compose([
    tt.Resize((224,224)),

    # Random Transformations
    tt.RandomHorizontalFlip(),
    tt.ColorJitter(
        brightness=0.2,
        contrast=0.2,
        saturation=0.2,
        hue=0.1
    ),
    tt.RandomRotation(100),

    tt.ToTensor(),
    tt.Normalize(mean, std)
])

# Transforms limited to resizing, converting to a tensor
# and normalization
test_transform = tt.Compose([
    tt.Resize((224,224)),
    tt.ToTensor(),
    tt.Normalize(mean, std)
])

```


1.4.4 Defining Datasets For Training and Testing

```
[36]: # Need to create the dataset objects
      # twice to properly create the test and training sets with the correct
      # transforms tied to them

      # Test dataset
      stanford_test = StanfordCars(
          root=path_stanford_cars,
          global_class_to_idx=global_class_to_idx,
          class_normalizer=normalize_classes,
          max_images_per_class = MAX_CLASSES,
          transform=test_transform
      )
      vmrdb_test = VMRDB_Dataset(
          root_dir=path_vmrdb,
          global_class_to_idx=global_class_to_idx,
          class_normalization=normalize_classes,
          max_images_per_class = MAX_CLASSES,
          transform=test_transform
      )

      # Train dataset
      stanford_train = StanfordCars(
          root=path_stanford_cars,
          global_class_to_idx=global_class_to_idx,
          class_normalizer=normalize_classes,
          max_images_per_class = MAX_CLASSES,
          transform=train_transform
      )
      vmrdb_train = VMRDB_Dataset(
          root_dir=path_vmrdb,
          global_class_to_idx=global_class_to_idx,
          class_normalization=normalize_classes,
          max_images_per_class = MAX_CLASSES,
          transform=train_transform
      )
```

1.4.5 Splitting the Training and Testing datasets

70% of the combined dataset allocated for the training set, 10% for the validation set and 20% for the testing set

```
[ ]: # Splitting into training and testing
      full = ConcatDataset([stanford_train, vmrdb_train])

      # 70 - 10 - 20 split
      train_size = int(0.70 * len(full))
```

```

val_size = int(0.10 * len(full))
test_size = len(full) - val_size - train_size

# Test + train + val set
generator = torch.Generator().manual_seed(0)
indices = torch.randperm(len(full), generator=generator)
train_indices = indices[:train_size]
val_indices = indices[train_size:train_size + val_size]
test_indices = indices[train_size + val_size:]

# Creating the datasets
dataset_train = Subset(ConcatDataset([stanford_train, vmrdb_train]),
    ↪train_indices)
dataset_val = Subset(ConcatDataset([stanford_test, vmrdb_test]), val_indices)
dataset_test = Subset(ConcatDataset([stanford_test, vmrdb_test]), test_indices)

# Creating the dataloaders
train_loader = DataLoader(dataset_train, BATCH_SIZE, num_workers = 0,
    ↪pin_memory = True, shuffle=True)
val_loader = DataLoader(dataset_val, BATCH_SIZE, num_workers = 0, pin_memory =
    ↪True, shuffle=False)
test_loader = DataLoader(dataset_test, BATCH_SIZE, num_workers = 0, pin_memory
    ↪= True, shuffle=False)

```

1.4.6 ResNet-50

Using the ResNet-50 Architecture to train a model that predicts model, make and year

```

[39]: class Bottleneck(nn.Module):
    expansion = 4
    def __init__(self, in_channels, out_channels, stride=1):
        super(Bottleneck, self).__init__()

        # 1x1 conv to reduce channels
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=1, stride=1,
    ↪padding=0, bias=False)
        self.batch_norm1 = nn.BatchNorm2d(out_channels)

        # 3x3 conv to process on reduced channels
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3,
    ↪stride=stride, padding=1, bias=False)
        self.batch_norm2 = nn.BatchNorm2d(out_channels)

        # 1x1 conv to expand channels
        self.conv3 = nn.Conv2d(out_channels, out_channels * self.expansion,
    ↪kernel_size=1, stride=1, padding=0, bias=False)
        self.batch_norm3 = nn.BatchNorm2d(out_channels * self.expansion)

```

```

    # Skip Connection
    self.shortcut = nn.Sequential()

    if stride != 1 or in_channels != out_channels * self.expansion:
        self.shortcut = nn.Sequential(
            nn.Conv2d(in_channels, out_channels * self.expansion, kernel_size=1,
↳ stride=stride, bias=False),
            nn.BatchNorm2d(out_channels * self.expansion)
        )

    def forward(self, x):

        # Channel reduction
        out = F.relu(self.batch_norm1(self.conv1(x)))
        # Process
        out = F.relu(self.batch_norm2(self.conv2(out)))
        # Channel expansion
        out = self.batch_norm3(self.conv3(out))
        # Residual connection
        out += self.shortcut(x)
        return F.relu(out)

class ResNet(nn.Module):
    def __init__(self, block, layers, num_classes=1000):
        super(ResNet, self).__init__()

        self.in_channels = 64
        self.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2,
                                padding=3, bias=False)
        self.bn1 = nn.BatchNorm2d(64)
        self.relu = nn.ReLU()
        self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)

        # ResNet layers
        self.layer1 = self._make_layer(block, 64, layers[0], stride=1)
        self.layer2 = self._make_layer(block, 128, layers[1], stride=2)
        self.layer3 = self._make_layer(block, 256, layers[2], stride=2)
        self.layer4 = self._make_layer(block, 512, layers[3], stride=2)

        self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
        self.fc = nn.Linear(512 * block.expansion, num_classes)

    def _make_layer(self, block, out_channels, blocks, stride):
        layers = []

        # First block

```

```

        layers.append(block(self.in_channels, out_channels, stride))
        self.in_channels = out_channels * block.expansion

        # Remaining blocks
        for _ in range(1, blocks):
            layers.append(block(self.in_channels, out_channels))

        return nn.Sequential(*layers)

    def forward(self, x):
        # First convolution step using 7x7
        x = self.relu(self.bn1(self.conv1(x)))
        x = self.maxpool(x)

        # ResNet Layers
        x = self.layer1(x)
        x = self.layer2(x)
        x = self.layer3(x)
        x = self.layer4(x)

        # Fully connected portion
        x = self.avgpool(x)
        x = torch.flatten(x, 1)
        x = self.fc(x)

        return x

def ResNet50(num_classes):
    return ResNet(Bottleneck, [3, 4, 6, 3], num_classes)

```

1.5 Training Code

```

[ ]: # HQNN training on top of frozen ResNet-50 backbone
    # pretrained model: ResNet-50 with weights trained on ImageNet-1k

    device = "cuda" if torch.cuda.is_available() else "cpu"
    print("device:", device)
    num_classes = len(global_class_to_idx)
    loss_criterion = nn.CrossEntropyLoss()

    class HybridHead(nn.Module):
        def __init__(self, in_dim: int, num_classes: int, n_qubits: int = 6,
            ↪ n_layers: int = 2, adapter_dim: int = 128):
            super().__init__()

```

```

        self.adapter = nn.Sequential(
            nn.Linear(in_dim, adapter_dim),
            nn.ReLU(inplace=True),
            nn.Linear(adapter_dim, n_qubits),
        )
        self.scale = nn.Parameter(torch.ones(n_qubits))

    dev = qml.device("default.qubit", wires=n_qubits)

    @qml.qnode(dev, interface="torch", diff_method="backprop")
    def circuit(inputs, weights):
        qml.AngleEmbedding(inputs, wires=range(n_qubits), rotation="X")
        qml.StronglyEntanglingLayers(weights, wires=range(n_qubits))
        return [qml.expval(qml.PauliZ(i)) for i in range(n_qubits)]

    self.qlayer = qml.qnn.TorchLayer(circuit, {"weights": (n_layers, n_qubits, 3)})
    self.classifier = nn.Linear(n_qubits, num_classes)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        h = self.adapter(x)
        angles = torch.tanh(h) * self.scale * torch.pi
        q_out = self.qlayer(angles)
        return self.classifier(q_out)

net = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2).to(device)
feature_dim = net.fc.in_features

# Freeze the backbone and replace final layer with identity for feature extraction
backbone_fc = net.fc
net.fc = nn.Identity()
for p in net.parameters():
    p.requires_grad = False

hqnn_head = HybridHead(in_dim=feature_dim, num_classes=num_classes, n_qubits=6, n_layers=2, adapter_dim=128).to(device)
optimizer = optim.AdamW(hqnn_head.parameters(), lr=1e-3, weight_decay=1e-4)

@torch.no_grad()
def cache_features(loader, model):
    model.eval()
    feats = []
    labels = []
    for X, y in tqdm(loader, leave=False, desc="Caching feats"):

```

```

        X = X.to(device, non_blocking=True)
        f = model(X).float().cpu()
        feats.append(f)
        labels.append(y.cpu())
    return torch.cat(feats, dim=0), torch.cat(labels, dim=0)

print("Caching train/val features from frozen ResNet backbone...")
train_feats, train_labels = cache_features(train_loader, net)
val_feats, val_labels = cache_features(val_loader, net)

hqnn_batch_size = 64
train_feat_loader = DataLoader(TensorDataset(train_feats, train_labels),
    ↪batch_size=hqnn_batch_size, shuffle=True)
val_feat_loader = DataLoader(TensorDataset(val_feats, val_labels),
    ↪batch_size=hqnn_batch_size, shuffle=False)

checkpoint_path = "./hqnn_checkpoint.pth"
best_checkpoint_path = "./hqnn_best_checkpoint.pth"
start_epoch = 0

train_losses = []
val_losses = []
val_accuracies = []
best_val_acc = 0.0

try:
    checkpoint = torch.load(checkpoint_path, map_location=device)
    hqnn_head.load_state_dict(checkpoint["model_state"])
    optimizer.load_state_dict(checkpoint["optimizer_state"])
    train_losses = checkpoint.get("train_losses", [])
    val_losses = checkpoint.get("val_losses", [])
    val_accuracies = checkpoint.get("val_accuracies", [])
    best_val_acc = checkpoint.get("best_val_acc", 0.0)
    start_epoch = checkpoint["epoch"] + 1
    print(f"Resuming HQNN from epoch {start_epoch}")
except FileNotFoundError:
    print("No HQNN checkpoint found, starting fresh.")

hqnn_epochs = min(epochs, 20)

for epoch in range(start_epoch, hqnn_epochs):
    hqnn_head.train()
    running_loss = 0.0
    num_batches = 0

    train_loop = tqdm(train_feat_loader, leave=True)

```

```

for Xf, y in train_loop:
    Xf = Xf.to(device, non_blocking=True)
    y = y.to(device, non_blocking=True)

    optimizer.zero_grad(set_to_none=True)
    output = hqnn_head(Xf)
    loss = loss_criterion(output, y)
    loss.backward()
    optimizer.step()

    running_loss += loss.item()
    num_batches += 1
    train_loop.set_description(f"HQNN Epoch [{epoch+1}/{hqnn_epochs}]")
    train_loop.set_postfix(loss=loss.item())

train_loss = running_loss / max(num_batches, 1)
train_losses.append(train_loss)

hqnn_head.eval()
val_loss = 0.0
val_batches = 0
correct = 0
total = 0

with torch.no_grad():
    val_loop = tqdm(val_feat_loader, leave=True, desc="HQNN Validation")
    for Xf, y in val_loop:
        Xf = Xf.to(device, non_blocking=True)
        y = y.to(device, non_blocking=True)

        output = hqnn_head(Xf)
        loss = loss_criterion(output, y)

        val_loss += loss.item()
        val_batches += 1
        pred = output.argmax(dim=1)
        correct += (pred == y).sum().item()
        total += y.size(0)

val_loss /= max(val_batches, 1)
val_acc = correct / max(total, 1)

val_losses.append(val_loss)
val_accuaries.append(val_acc)

print(f"HQNN epoch {epoch+1}: train_loss={train_loss:.4f},
↪ val_loss={val_loss:.4f}, val_acc={val_acc:.4f}")

```

```

checkpoint = {
    "epoch": epoch,
    "model_state": hqnn_head.state_dict(),
    "optimizer_state": optimizer.state_dict(),
    "train_losses": train_losses,
    "val_losses": val_losses,
    "val_accuracies": val_accuracies,
    "best_val_acc": best_val_acc,
}
torch.save(checkpoint, checkpoint_path)

if val_acc > best_val_acc:
    best_val_acc = val_acc
    checkpoint["best_val_acc"] = best_val_acc
    torch.save(checkpoint, best_checkpoint_path)

print(f"Finished HQNN training. Best val accuracy: {best_val_acc:.4f}")

# Restore classical head
net.fc = backbone_fc

```

device: cuda

Caching train/val features from frozen ResNet backbone...

No HQNN checkpoint found, starting fresh.

HQNN Epoch [1/20]: 100%| | 1691/1691 [01:34<00:00, 17.82it/s,
loss=7.27]

HQNN Validation: 100%| | 242/242 [00:05<00:00, 40.94it/s]

HQNN epoch 1: train_loss=7.4961, val_loss=7.3285, val_acc=0.0012

HQNN Epoch [2/20]: 100%| | 1691/1691 [01:31<00:00, 18.56it/s,
loss=6.84]

HQNN Validation: 100%| | 242/242 [00:05<00:00, 46.04it/s]

HQNN epoch 2: train_loss=7.1731, val_loss=6.9261, val_acc=0.0032

HQNN Epoch [3/20]: 100%| | 1691/1691 [01:23<00:00, 20.15it/s,
loss=6.49]

HQNN Validation: 100%| | 242/242 [00:04<00:00, 54.13it/s]

HQNN epoch 3: train_loss=6.7971, val_loss=6.7177, val_acc=0.0045

HQNN Epoch [4/20]: 100%| | 1691/1691 [01:32<00:00, 18.19it/s,
loss=6.24]

HQNN Validation: 100%| | 242/242 [00:05<00:00, 42.00it/s]

HQNN epoch 4: train_loss=6.6388, val_loss=6.6027, val_acc=0.0050

HQNN Epoch [5/20]: 100%| | 1691/1691 [01:28<00:00, 19.07it/s, loss=6.3]
 HQNN Validation: 100%| | 242/242 [00:04<00:00, 50.93it/s]
 HQNN epoch 5: train_loss=6.5475, val_loss=6.5613, val_acc=0.0045
 HQNN Epoch [6/20]: 100%| | 1691/1691 [01:26<00:00, 19.56it/s, loss=6.56]
 HQNN Validation: 100%| | 242/242 [00:04<00:00, 51.29it/s]
 HQNN epoch 6: train_loss=6.4824, val_loss=6.5079, val_acc=0.0049
 HQNN Epoch [7/20]: 100%| | 1691/1691 [01:31<00:00, 18.49it/s, loss=6.34]
 HQNN Validation: 100%| | 242/242 [00:05<00:00, 46.29it/s]
 HQNN epoch 7: train_loss=6.4307, val_loss=6.4949, val_acc=0.0047
 HQNN Epoch [8/20]: 100%| | 1691/1691 [01:32<00:00, 18.19it/s, loss=6.35]
 HQNN Validation: 100%| | 242/242 [00:05<00:00, 47.64it/s]
 HQNN epoch 8: train_loss=6.3867, val_loss=6.4580, val_acc=0.0055
 HQNN Epoch [9/20]: 100%| | 1691/1691 [01:33<00:00, 18.00it/s, loss=6.53]
 HQNN Validation: 100%| | 242/242 [00:05<00:00, 40.51it/s]
 HQNN epoch 9: train_loss=6.3475, val_loss=6.4431, val_acc=0.0050
 HQNN Epoch [10/20]: 100%| | 1691/1691 [01:32<00:00, 18.28it/s, loss=6.15]
 HQNN Validation: 100%| | 242/242 [00:04<00:00, 56.18it/s]
 HQNN epoch 10: train_loss=6.3141, val_loss=6.4261, val_acc=0.0057
 HQNN Epoch [11/20]: 100%| | 1691/1691 [01:14<00:00, 22.72it/s, loss=6.11]
 HQNN Validation: 100%| | 242/242 [00:04<00:00, 56.93it/s]
 HQNN epoch 11: train_loss=6.2841, val_loss=6.4190, val_acc=0.0050
 HQNN Epoch [12/20]: 100%| | 1691/1691 [01:15<00:00, 22.48it/s, loss=6.34]
 HQNN Validation: 100%| | 242/242 [00:04<00:00, 56.95it/s]
 HQNN epoch 12: train_loss=6.2584, val_loss=6.4238, val_acc=0.0045
 HQNN Epoch [13/20]: 100%| | 1691/1691 [01:14<00:00, 22.75it/s, loss=6.14]
 HQNN Validation: 100%| | 242/242 [00:04<00:00, 56.57it/s]
 HQNN epoch 13: train_loss=6.2335, val_loss=6.4135, val_acc=0.0055
 HQNN Epoch [14/20]: 100%| | 1691/1691 [01:14<00:00, 22.62it/s, loss=6.43]
 HQNN Validation: 100%| | 242/242 [00:04<00:00, 56.69it/s]

```

HQNN epoch 14: train_loss=6.2112, val_loss=6.4090, val_acc=0.0048
HQNN Epoch [15/20]: 100%|      | 1691/1691 [01:14<00:00, 22.79it/s,
loss=6.28]
HQNN Validation: 100%|      | 242/242 [00:04<00:00, 57.19it/s]

HQNN epoch 15: train_loss=6.1907, val_loss=6.3988, val_acc=0.0052
HQNN Epoch [16/20]: 100%|      | 1691/1691 [01:13<00:00, 22.85it/s,
loss=5.79]
HQNN Validation: 100%|      | 242/242 [00:04<00:00, 57.20it/s]

HQNN epoch 16: train_loss=6.1714, val_loss=6.4147, val_acc=0.0051
HQNN Epoch [17/20]: 100%|      | 1691/1691 [01:14<00:00, 22.76it/s,
loss=6.11]
HQNN Validation: 100%|      | 242/242 [00:04<00:00, 57.01it/s]

HQNN epoch 17: train_loss=6.1553, val_loss=6.4005, val_acc=0.0061
HQNN Epoch [18/20]: 100%|      | 1691/1691 [01:14<00:00, 22.76it/s,
loss=6.18]
HQNN Validation: 100%|      | 242/242 [00:04<00:00, 57.02it/s]

HQNN epoch 18: train_loss=6.1368, val_loss=6.3969, val_acc=0.0057
HQNN Epoch [19/20]: 100%|      | 1691/1691 [01:14<00:00, 22.78it/s,
loss=6.1]
HQNN Validation: 100%|      | 242/242 [00:04<00:00, 57.15it/s]

HQNN epoch 19: train_loss=6.1208, val_loss=6.3986, val_acc=0.0052
HQNN Epoch [20/20]: 100%|      | 1691/1691 [01:14<00:00, 22.76it/s,
loss=6.08]
HQNN Validation: 100%|      | 242/242 [00:04<00:00, 56.52it/s]

HQNN epoch 20: train_loss=6.1064, val_loss=6.4021, val_acc=0.0050
Finished HQNN training. Best val accuracy: 0.0061

```

```

[ ]: # HQNN fine-tuning with partial backbone unfreezing + top-5 metrics
     # Trains end-to-end (no cached features) so unfrozen layers can actually learn

device = "cuda" if torch.cuda.is_available() else "cpu"
print("device:", device)
num_classes = len(global_class_to_idx)
loss_criterion = nn.CrossEntropyLoss(label_smoothing=0.1)

class HybridHead(nn.Module):
    def __init__(self, in_dim: int, num_classes: int, n_qubits: int = 6,
        n_layers: int = 2, adapter_dim: int = 128):
        super().__init__()

```

```

self.adapter = nn.Sequential(
    nn.Linear(in_dim, adapter_dim),
    nn.ReLU(inplace=True),
    nn.Linear(adapter_dim, n_qubits),
)
self.scale = nn.Parameter(torch.ones(n_qubits))

dev = qml.device("default.qubit", wires=n_qubits)

@qml.qnode(dev, interface="torch", diff_method="backprop")
def circuit(inputs, weights):
    qml.AngleEmbedding(inputs, wires=range(n_qubits), rotation="X")
    qml.StronglyEntanglingLayers(weights, wires=range(n_qubits))
    return [qml.expval(qml.PauliZ(i)) for i in range(n_qubits)]

self.qlayer = qml.qnn.TorchLayer(circuit, {"weights": (n_layers, n_qubits, 3)})
self.classifier = nn.Linear(n_qubits, num_classes)

def forward(self, x: torch.Tensor) -> torch.Tensor:
    h = self.adapter(x)
    angles = torch.tanh(h) * self.scale * torch.pi
    q_out = self.qlayer(angles)
    return self.classifier(q_out)

# Build pretrained backbone and expose penultimate features.
net = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2).to(device)
feature_dim = net.fc.in_features
net.fc = nn.Identity()

# Freeze everything, then unfreeze only layer4 for partial fine-tuning.
for p in net.parameters():
    p.requires_grad = False
for p in net.layer4.parameters():
    p.requires_grad = True

hqnn_head = HybridHead(in_dim=feature_dim, num_classes=num_classes, n_qubits=6, n_layers=2, adapter_dim=128).to(device)

# Different learning rates for backbone vs HQNN head.
optimizer = optim.AdamW(
    [
        {"params": net.layer4.parameters(), "lr": 1e-5, "weight_decay": 1e-4},
        {"params": hqnn_head.parameters(), "lr": 1e-3, "weight_decay": 1e-4},
    ]
)

```

```

scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=min(epochs,
↪20))

checkpoint_path = "./hqnn_unfreeze_checkpoint.pth"
best_checkpoint_path = "./hqnn_unfreeze_best_checkpoint.pth"
start_epoch = 0

train_losses = []
val_losses = []
val_top1_accuracies = []
val_top5_accuracies = []
best_val_top1 = 0.0

try:
    checkpoint = torch.load(checkpoint_path, map_location=device)
    net.load_state_dict(checkpoint["backbone_state"])
    hqnn_head.load_state_dict(checkpoint["model_state"])
    optimizer.load_state_dict(checkpoint["optimizer_state"])
    scheduler.load_state_dict(checkpoint["scheduler_state"])
    train_losses = checkpoint.get("train_losses", [])
    val_losses = checkpoint.get("val_losses", [])
    val_top1_accuracies = checkpoint.get("val_top1_accuracies", [])
    val_top5_accuracies = checkpoint.get("val_top5_accuracies", [])
    best_val_top1 = checkpoint.get("best_val_top1", 0.0)
    start_epoch = checkpoint["epoch"] + 1
    print(f"Resuming HQNN+unfreeze from epoch {start_epoch}")
except FileNotFoundError:
    print("No HQNN+unfreeze checkpoint found, starting fresh.")

hqnn_epochs = min(epochs, 20)

for epoch in range(start_epoch, hqnn_epochs):
    net.train()
    hqnn_head.train()

    running_loss = 0.0
    num_batches = 0

    train_loop = tqdm(train_loader, leave=True)
    for X, y in train_loop:
        X = X.to(device, non_blocking=True)
        y = y.to(device, non_blocking=True)

        optimizer.zero_grad(set_to_none=True)
        feats = net(X)
        output = hqnn_head(feats)

```

```

        loss = loss_criterion(output, y)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        num_batches += 1
        train_loop.set_description(f"HQNN+FT Epoch [{epoch+1}/{hqnn_epochs}]")
        train_loop.set_postfix(loss=loss.item())

train_loss = running_loss / max(num_batches, 1)
train_losses.append(train_loss)

net.eval()
hqnn_head.eval()

val_loss = 0.0
val_batches = 0
top1_correct = 0
top5_correct = 0
total = 0

with torch.no_grad():
    val_loop = tqdm(val_loader, leave=True, desc="HQNN+FT Validation")
    for X, y in val_loop:
        X = X.to(device, non_blocking=True)
        y = y.to(device, non_blocking=True)

        feats = net(X)
        output = hqnn_head(feats)
        loss = loss_criterion(output, y)

        val_loss += loss.item()
        val_batches += 1

        pred_top1 = output.argmax(dim=1)
        top1_correct += (pred_top1 == y).sum().item()

        topk_idx = output.topk(k=5, dim=1).indices
        top5_correct += topk_idx.eq(y.view(-1, 1)).any(dim=1).sum().item()
        total += y.size(0)

val_loss /= max(val_batches, 1)
val_top1 = top1_correct / max(total, 1)
val_top5 = top5_correct / max(total, 1)

val_losses.append(val_loss)
val_top1_accuracies.append(val_top1)

```

```

val_top5_accuracies.append(val_top5)

print(
    f"HQNN+FT epoch {epoch+1}: "
    f"train_loss={train_loss:.4f}, val_loss={val_loss:.4f}, "
    f"val_top1={val_top1:.4f}, val_top5={val_top5:.4f}"
)

scheduler.step()

checkpoint = {
    "epoch": epoch,
    "backbone_state": net.state_dict(),
    "model_state": hqnn_head.state_dict(),
    "optimizer_state": optimizer.state_dict(),
    "scheduler_state": scheduler.state_dict(),
    "train_losses": train_losses,
    "val_losses": val_losses,
    "val_top1_accuracies": val_top1_accuracies,
    "val_top5_accuracies": val_top5_accuracies,
    "best_val_top1": best_val_top1,
}
torch.save(checkpoint, checkpoint_path)

if val_top1 > best_val_top1:
    best_val_top1 = val_top1
    checkpoint["best_val_top1"] = best_val_top1
    torch.save(checkpoint, best_checkpoint_path)

print(f"Finished HQNN+FT training. Best val top-1 accuracy: {best_val_top1:.4f}")

```

device: cuda

No HQNN+unfreeze checkpoint found, starting fresh.

HQNN+FT Epoch [1/20]: 100%| | 6762/6762 [11:15<00:00, 10.00it/s, loss=7.26]

HQNN+FT Validation: 100%| | 966/966 [01:06<00:00, 14.52it/s]

HQNN+FT epoch 1: train_loss=7.3942, val_loss=7.0705, val_top1=0.0019, val_top5=0.0136

HQNN+FT Epoch [2/20]: 100%| | 6762/6762 [10:51<00:00, 10.38it/s, loss=7.41]

HQNN+FT Validation: 100%| | 966/966 [01:01<00:00, 15.82it/s]

HQNN+FT epoch 2: train_loss=6.9821, val_loss=6.8734, val_top1=0.0038, val_top5=0.0162

HQNN+FT Epoch [3/20]: 100%| | 6762/6762 [10:50<00:00, 10.39it/s,

```

loss=6.82]
HQNN+FT Validation: 100%|      | 966/966 [01:03<00:00, 15.24it/s]

HQNN+FT epoch 3: train_loss=6.8409, val_loss=6.7766, val_top1=0.0045,
val_top5=0.0208

HQNN+FT Epoch [4/20]: 100%|      | 6762/6762 [11:05<00:00, 10.15it/s,
loss=6.1]
HQNN+FT Validation: 100%|      | 966/966 [01:01<00:00, 15.79it/s]

HQNN+FT epoch 4: train_loss=6.7426, val_loss=6.6934, val_top1=0.0061,
val_top5=0.0247

HQNN+FT Epoch [5/20]: 100%|      | 6762/6762 [11:04<00:00, 10.18it/s,
loss=6.48]
HQNN+FT Validation: 100%|      | 966/966 [01:01<00:00, 15.63it/s]

HQNN+FT epoch 5: train_loss=6.6609, val_loss=6.6163, val_top1=0.0063,
val_top5=0.0267

HQNN+FT Epoch [6/20]: 100%|      | 6762/6762 [11:02<00:00, 10.21it/s,
loss=6.3]
HQNN+FT Validation: 100%|      | 966/966 [01:02<00:00, 15.36it/s]

HQNN+FT epoch 6: train_loss=6.5821, val_loss=6.5510, val_top1=0.0063,
val_top5=0.0288

HQNN+FT Epoch [7/20]: 100%|      | 6762/6762 [10:53<00:00, 10.35it/s,
loss=5.99]
HQNN+FT Validation: 100%|      | 966/966 [01:01<00:00, 15.59it/s]

HQNN+FT epoch 7: train_loss=6.5176, val_loss=6.5008, val_top1=0.0074,
val_top5=0.0293

HQNN+FT Epoch [8/20]: 100%|      | 6762/6762 [10:50<00:00, 10.40it/s,
loss=6.84]
HQNN+FT Validation: 100%|      | 966/966 [01:01<00:00, 15.71it/s]

HQNN+FT epoch 8: train_loss=6.4667, val_loss=6.4596, val_top1=0.0077,
val_top5=0.0340

HQNN+FT Epoch [9/20]: 100%|      | 6762/6762 [10:49<00:00, 10.41it/s,
loss=6.48]
HQNN+FT Validation: 100%|      | 966/966 [01:01<00:00, 15.78it/s]

HQNN+FT epoch 9: train_loss=6.4250, val_loss=6.4250, val_top1=0.0074,
val_top5=0.0321

HQNN+FT Epoch [10/20]: 100%|      | 6762/6762 [10:48<00:00, 10.43it/s,
loss=6.16]
HQNN+FT Validation: 100%|      | 966/966 [01:02<00:00, 15.35it/s]

HQNN+FT epoch 10: train_loss=6.3894, val_loss=6.3974, val_top1=0.0079,
val_top5=0.0366

```

HQNN+FT Epoch [11/20]: 100%| | 6762/6762 [10:47<00:00, 10.44it/s, loss=5.81]
HQNN+FT Validation: 100%| | 966/966 [01:01<00:00, 15.74it/s]
HQNN+FT epoch 11: train_loss=6.3610, val_loss=6.3781, val_top1=0.0079, val_top5=0.0362

HQNN+FT Epoch [12/20]: 100%| | 6762/6762 [10:52<00:00, 10.36it/s, loss=6.51]
HQNN+FT Validation: 100%| | 966/966 [01:03<00:00, 15.25it/s]
HQNN+FT epoch 12: train_loss=6.3388, val_loss=6.3623, val_top1=0.0083, val_top5=0.0369

HQNN+FT Epoch [13/20]: 100%| | 6762/6762 [11:03<00:00, 10.19it/s, loss=7.33]
HQNN+FT Validation: 100%| | 966/966 [01:04<00:00, 14.99it/s]
HQNN+FT epoch 13: train_loss=6.3183, val_loss=6.3523, val_top1=0.0084, val_top5=0.0397

HQNN+FT Epoch [14/20]: 100%| | 6762/6762 [11:05<00:00, 10.15it/s, loss=6.36]
HQNN+FT Validation: 100%| | 966/966 [01:04<00:00, 14.87it/s]
HQNN+FT epoch 14: train_loss=6.3017, val_loss=6.3446, val_top1=0.0098, val_top5=0.0400

HQNN+FT Epoch [15/20]: 100%| | 6762/6762 [11:04<00:00, 10.18it/s, loss=6.4]
HQNN+FT Validation: 100%| | 966/966 [01:04<00:00, 15.07it/s]
HQNN+FT epoch 15: train_loss=6.2895, val_loss=6.3297, val_top1=0.0096, val_top5=0.0413

HQNN+FT Epoch [16/20]: 100%| | 6762/6762 [10:56<00:00, 10.29it/s, loss=6.02]
HQNN+FT Validation: 100%| | 966/966 [01:01<00:00, 15.76it/s]
HQNN+FT epoch 16: train_loss=6.2810, val_loss=6.3300, val_top1=0.0098, val_top5=0.0421

HQNN+FT Epoch [17/20]: 100%| | 6762/6762 [10:51<00:00, 10.38it/s, loss=7.01]
HQNN+FT Validation: 100%| | 966/966 [01:02<00:00, 15.50it/s]
HQNN+FT epoch 17: train_loss=6.2727, val_loss=6.3267, val_top1=0.0100, val_top5=0.0406

HQNN+FT Epoch [18/20]: 100%| | 6762/6762 [10:55<00:00, 10.32it/s, loss=6.15]
HQNN+FT Validation: 100%| | 966/966 [01:04<00:00, 15.03it/s]
HQNN+FT epoch 18: train_loss=6.2691, val_loss=6.3240, val_top1=0.0101, val_top5=0.0421


```
HQNN+FT Epoch [19/20]: 100%|      | 6762/6762 [10:52<00:00, 10.37it/s,
loss=6.26]
HQNN+FT Validation: 100%|      | 966/966 [01:04<00:00, 15.00it/s]
HQNN+FT epoch 19: train_loss=6.2661, val_loss=6.3248, val_top1=0.0102,
val_top5=0.0412
HQNN+FT Epoch [20/20]: 100%|      | 6762/6762 [11:01<00:00, 10.22it/s,
loss=6.47]
HQNN+FT Validation: 100%|      | 966/966 [01:04<00:00, 14.88it/s]
HQNN+FT epoch 20: train_loss=6.2643, val_loss=6.3259, val_top1=0.0102,
val_top5=0.0417
Finished HQNN+FT training. Best val top-1 accuracy: 0.0102
```